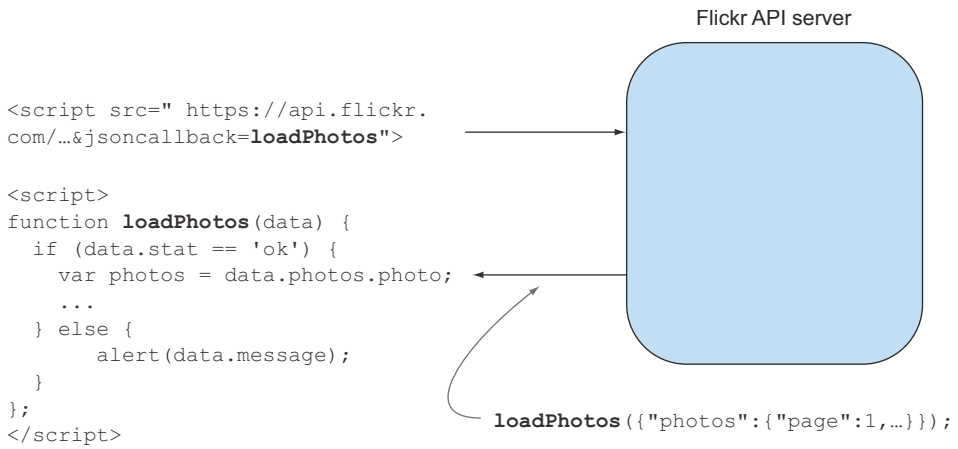


Figure D.3 Making a cross-origin request using JSONP

callback function (this is where the word padding in JSON with padding comes from). For example, if the API response looks like this:

```
{ "photos": { "page": 1, "photo": [...] } }
```

JSONP would wrap the response in a callback, like this:

```
loadPhotos({ "photos": { "page": 1, "photo": [...] } });
```

Finally, when the browser receives the server's response, it actually calls the callback function with the data provided by the server. In this example, the `loadPhotos` function defined by the client gets called with the API response data.

JSONP is one of the oldest cross-origin techniques out there, and it enjoys wide browser support. But because it uses a `<script>` tag, JSONP can only support `GET` requests. There is no way to make a `POST`, `PUT`, or `DELETE` request using JSONP. If the browser has any cookies from the server, those cookies will be included on the request. This can open up the API to cross-site forgery requests.

JSONP is ideal for sharing publicly available data. The preceding Flickr example is a perfect use case for JSONP: the images are publicly available, and the sample is only displaying the images on the page. Due to the cross-site forgery concerns, JSONP shouldn't be used to make updates to data, or to share sensitive user data. Finally, because the server is essentially executing arbitrary JavaScript in the user's browser, it's important that the client fully trust the response from the server.

D.2 Flash

Adobe's Flash is a software platform that brings rich, interactive applications to the browser. Flash applications are written in a language called ActionScript, which can make cross-origin requests to a remote server, as shown in figure D.4.

Similar to CORS, Flash requires the server to grant permission to clients making cross-origin requests. Except instead of HTTP headers, Flash uses an XML file that lives